

NLP Labs

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab') # Download the punkt_tab data
print(nltk.data.path)
```

Q1. Write a Python program to perform Part of Speech Tagging with Stop words using NLTK.

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk import pos_tag

text = "The quick brown fox jumped over the lazy dog"
tokens = word_tokenize(text)
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]
tagged_tokens = pos_tag(filtered_tokens)

print("Filtered Tokens:", filtered_tokens)
print("POS Tagging with NLTK:", tagged_tokens)
```

Q2 Write a Python program to find Term Frequency and Inverse Document Frequency (TF-IDF)

```
# !pip install scikit-learn
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

documents = [
    "The quick brown fox jumped over the lazy dog",
    "The dog barked at the fox",
    "A fox and a dog are playing in the yard",
]
```

```

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(documents)
tfidf_df = pd.DataFrame(tfidf_matrix.toarray(), columns=vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", tfidf_df)
print("\nIDF Values:\n", vectorizer.idf_)

```

Q3. Implement word embedding using Word2Vec/Glove/fast using Python

```

# word to vect : !pip install gensim
# Download necessary NLTK resources- nltk.download('punkt_tab')
import nltk
import gensim
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize

```

text = "Word2vec is a technique in natural language processing (NLP) for obtaining vector representations of words. These vectors capture information about the meaning of the word based on the surrounding words. The word2vec algorithm estimates these representations by modeling text in a large corpus. "

```

tokens = word_tokenize(text.lower())
model = Word2Vec([tokens], vector_size=10, window=5, min_count=1, workers=4)
model.save("word2vec.model")
model = Word2Vec.load("word2vec.model")
vector = model.wv['word2vec']
print(f"Vector for 'word2vec': {vector}")
similar_words = model.wv.most_similar('word2vec', topn=3)
print(f"Most similar words to 'word2vec': {similar_words}")

```

Q4 Implement N-gram Language model using Python

```

from nltk.util import ngrams
from nltk.tokenize import word_tokenize
from collections import Counter, defaultdict

sentence = "The quick brown fox jumps over the lazy dog"
tokens = word_tokenize(sentence)

```

```

n = 4 # Change this to 2 for bigrams, 3 for trigrams, etc.
ngrams_list = list(ngrams(tokens, n))
counts = Counter(ngrams_list)
model = defaultdict(float)
for ngram, count in counts.items():
    prefix = ngram[:-1]
    total_prefix_counts = sum(c for ng, c in counts.items() if ng[:-1] == prefix)
    model[ngram] = count / total_prefix_counts
print(f"{n}-grams and Probabilities:")
for ngram, prob in model.items():
    print(f"{ngram}: {prob:.4f}")

```

Q5 Implement text processing with LSTM. (Use the model to predict the next words in a sequence.)

```

import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

text = "Natural Language Processing with LSTM is fascinating and powerful. LSTM is used for sequence prediction."

tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
word_index = tokenizer.word_index
total_words = len(word_index) + 1
sequences = []

for i in range(1, len(tokenizer.texts_to_sequences([text])[0])):
    n_gram_sequence = tokenizer.texts_to_sequences([text])[0][i+1:]
    sequences.append(n_gram_sequence)

max_len = max([len(x) for x in sequences])
sequences = pad_sequences(sequences, maxlen=max_len, padding='pre')
X, y = sequences[:, :-1], sequences[:, -1]
y = np.eye(total_words)[y] # One-hot encode the output labels

```

```

model = Sequential([
    Embedding(total_words, 10, input_length=X.shape[1]),
    LSTM(50),
    Dense(total_words, activation='softmax')
])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=100, verbose=1)

```

```

seed_text = "Natural Language"
for _ in range(5): # Predict the next 5 words
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    token_list = pad_sequences([token_list], maxlen=X.shape[1], padding='pre')
    predicted = np.argmax(model.predict(token_list), axis=1)[0]
    for word, index in word_index.items():
        if index == predicted:
            seed_text += " " + word
            break
print("Generated Text:", seed_text)

```

Q6 Build a small-scale GPT model for text generation by creating a transformer-based model using PyTorch or TensorFlow, training it on a small dataset, and evaluating the generated text for coherence.

```

import torch

from transformers import GPT2Tokenizer, GPT2LMHeadModel, AdamW

tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")

text_data = """
Ella's adventure led her to shimmering lakes where the water whispered secrets.
In another village, Leo found an enchanted sword that only he could wield.
The forest was alive with mythical creatures—dragons, unicorns, and fairies guarding treasures.
"""

inputs = tokenizer(text_data, return_tensors="pt", max_length=512, truncation=True)
input_ids = inputs["input_ids"]

optimizer = AdamW(model.parameters(), lr=5e-5)

model.train()

for epoch in range(3):

```

```

outputs = model(input_ids, labels=input_ids)
loss = outputs.loss
loss.backward()
optimizer.step()
optimizer.zero_grad()
print(f"Epoch {epoch + 1}, Loss: {loss.item()}")

def generate_text(prompt, max_length=50):
    input_ids = tokenizer(prompt, return_tensors="pt")["input_ids"]
    outputs = model.generate(input_ids, max_length=max_length, temperature=0.8, repetition_penalty=1.2)
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

prompt = "Ella stumbled upon the crystal and wished for"
print("Generated Text:", generate_text(prompt))

```

Q7 Implement self-attention from scratch by creating a small sequence dataset, computing attention weights and outputs manually, and visualizing the attention map.

```

import numpy as np
import matplotlib.pyplot as plt

sequence = np.array([[1, 0, 1, 0], [0, 2, 0, 2], [1, 1, 1, 1]])

def self_attention(sequence):
    Q = K = V = sequence # Queries, Keys, Values
    scores = np.dot(Q, K.T) / np.sqrt(sequence.shape[-1]) # Scaled dot-product
    attention_weights = np.exp(scores) / np.exp(scores).sum(axis=-1, keepdims=True) # Softmax
    output = np.dot(attention_weights, V)
    return attention_weights, output

def plot_attention_map(weights):
    plt.imshow(weights) # cmap='viridis', aspect='auto')
    plt.colorbar()
    plt.title("Attention Map")
    plt.show()

attention_weights, output = self_attention(sequence)
print("Attention Weights:\n", attention_weights)
print("Attention Output:\n", output)
plot_attention_map(attention_weights)

```